

Worksheet

Computer Network - Week 8 - Process Synchronization

Name	Farrel Tsaqif Anindyo
Class	CSA

Activity 1

Procedure

Read the procedure on the activity file.

```
farrelta@ROG: ~/Downloads
(farrelta@ROG)~[/Downloads]
$ touch race.c && nano race.c
(farrelta@ROG)~[/Downloads]
$ gcc -pthread race.c -o race.out
(farrelta@ROG)~[/Downloads]
$ ./race.out
Initial value: 5
[THREAD 1] READ: 5
[THREAD 2] READ: 5
[THREAD 1] MODIFY: 5 -> 6
[THREAD 2] MODIFY: 5 -> 4
[THREAD 1] WRITE: 6
[THREAD 2] WRITE: 4
Final value: 4 (Expected: 5)
(farrelta@ROG)~[/Downloads]
$ ./race.out
Initial value: 5
[THREAD 1] READ: 5
[THREAD 2] READ: 5
[THREAD 1] MODIFY: 5 -> 6
[THREAD 2] MODIFY: 5 -> 4
[THREAD 1] WRITE: 6
[THREAD 2] WRITE: 4
Final value: 4 (Expected: 5)
```

Answers

1. Identify which lines of code form the critical section and explain why these lines require synchronization.

the critical section is the group of statements that access or modify the shared variable "shared_counter" which is "increment()" and "decrement()" Both lines require synchronization because both threads access the same shared memory "shared_counter" concurrently

2. Trace the execution using your output. Explain why both threads read the same value, and how that led to incorrect and inconsistent results across program runs.

Because the shared variable wasn't updated yet when each thread performed its read. Because of *race conditions* the two threads execute the read-modify-write sequence concurrently, interleaving unpredictably. The final outputs can come from either thread..

3. Differentiate concurrency vs. parallelism, state which one this program demonstrates, and justify whether the race condition can arise on a single-core system (consider time-slicing and preemption)

This program demonstrates concurrency and potential parallelism if executed on a multi-core processor.
Yes, race conditions can occur because of time-slicing and preemption.

Activity 2

Procedure

Read the procedure on the activity file.

```
farrelta@ROG: ~/Downloads
(farrelta@ROG)~[/Downloads]
$ cp race.c mutex.c
(farrelta@ROG)~[/Downloads]
$ nano mutex.c
(farrelta@ROG)~[/Downloads]
$ nano mutex.c
(farrelta@ROG)~[/Downloads]
$ gcc -o mutex.out mutex.c -lpthread
(farrelta@ROG)~[/Downloads]
$ ./mutex.out
Initial value: 5

[THREAD 1] Lock acquired
[THREAD 1] READ: 5
[THREAD 1] MODIFY: 5 -> 6
[THREAD 1] WRITE: 6
[THREAD 1] Lock released
[THREAD 2] Lock acquired
[THREAD 2] READ: 6
[THREAD 2] MODIFY: 6 -> 5
[THREAD 2] WRITE: 5
[THREAD 2] Lock released

Final value: 5 (Expected: 5)
```

```
farrelta@ROG: ~/Downloads
(farrelta@ROG)~[/Downloads]
$ nano mutex.c
(farrelta@ROG)~[/Downloads]
$ gcc -o mutex.out mutex.c -lpthread
(farrelta@ROG)~[/Downloads]
$ ./mutex.out
Initial value: 5

[THREAD 2] Lock acquired
[THREAD 2] READ: 5
[THREAD 1] Lock acquired
[THREAD 1] READ: 5
[THREAD 1] MODIFY: 5 -> 6
[THREAD 2] MODIFY: 5 -> 4
[THREAD 1] WRITE: 6
[THREAD 1] Lock released
[THREAD 2] WRITE: 4
[THREAD 2] Lock released

Final value: 4 (Expected: 5)
(farrelta@ROG)~[/Downloads]
$ |
```

Answers

1. Based on your working mutex output, prove that mutual exclusion was achieved. Which evidence from the output shows only one thread was in the critical section at a time?

When the lock was acquired, only the first thread that read, modified, and wrote the values that the mutual exclusion was achieved.

2. Omitting mutex unlock in Thread 1, describe the program's observable behavior, identify Thread 2's state, and explain why it cannot proceed (ownership + blocking).

Omitting `pthread_mutex_unlock()` in Thread 1 causes the mutex to remain locked. Since a mutex enforces exclusive ownership, Thread 2 cannot acquire it and is forced into a blocked state, waiting for a release that never occurs.

3. Contrast busy-waiting vs blocking in thread synchronization. Which does `pthread_mutex_lock` use in our setup, how would CPU usage differ, and when might a spinlock be preferable?

the "`pthread_mutex_lock()`" uses blocking synchronization. If a thread can't acquire the lock, it's put to sleep by the OS, not left spinning. CPU usage stays low since blocked threads don't burn CPU time. Spinlocks (busy-waiting) are only better when the critical section is extremely short and the cost of context switching is higher than a few spins

Activity 3

Procedure

Read the procedure on the activity file.

```
farrelta@ROG: ~/Downloads
(farrelta@ROG)~/Downloads
$ ./semaphore.out
Initial value: 5
[THREAD 1] Lock acquired
[THREAD 1] READ: 5
[THREAD 1] MODIFY: 5 -> 6
[THREAD 1] WRITE: 6
[THREAD 1] Lock released
[THREAD 2] Lock acquired
[THREAD 2] READ: 6
[THREAD 2] MODIFY: 6 -> 5
[THREAD 2] WRITE: 5
[THREAD 2] Lock released
Final value: 5 (Expected: 5)
(farrelta@ROG)~/Downloads
$ nano resource_pool.c
(farrelta@ROG)~/Downloads
$ gcc -o resource_pool.out resource_pool.c -lpthread
(farrelta@ROG)~/Downloads
$ ./resource_pool.out
Resource Pool: 2 available
Requests: 5 threads
[Thread 1] Requesting resource...
[Thread 1] Resource acquired
[Thread 2] Requesting resource...
[Thread 2] Resource acquired
[Thread 3] Requesting resource...
[Thread 4] Requesting resource...
[Thread 5] Requesting resource...
[Thread 1] Released resource
[Thread 3] Resource acquired
[Thread 2] Released resource
[Thread 4] Resource acquired
[Thread 3] Released resource
[Thread 5] Resource acquired
[Thread 4] Released resource
[Thread 5] Released resource
All completed!
```

Answers

1. In Part A, you replaced mutex with a binary semaphore (initial value = 1). What is the main difference between mutex and binary semaphore? (Hint: thinking about “ownership”)

Mutex Ownership:

A mutex has an owner — the thread that successfully locked it. So that only the thread that locked the mutex is allowed to unlock it. If another thread tries to unlock it, that’s an error (undefined behavior).

Binary Semaphore Ownership:

A binary semaphore does not have ownership. So that any thread can “sem_wait()” (decrement)

and any thread can “sem_post()” (increment), regardless of which thread acquired it.

2. The resource pool program uses `sem_init(&pool, 0, 2)`. When Threads 1 and 2 have acquired the semaphore, what happens when Thread 3 calls `sem_wait()`? Identify Thread 3's state and argue why this shows a counting semaphore (with 2 capacity) rather than a binary semaphore.

When threads 1 and 2 hold the semaphore, thread 3 “sem_wait()” call blocks because the semaphore value has reached zero. This blocking behavior, while still allowing two concurrent holders, is the defining property of a counting semaphore, as opposed to a binary semaphore, which can only represent a single available resource.

```
farrelta@ROG: ~/Downloads
[Thread 4] Released resource
[Thread 5] Released resource
All completed!
(farrelta@ROG)~/Downloads
$ nano resource_pool.c
(farrelta@ROG)~/Downloads
$ gcc -o resource_pool.out resource_pool.c -lpthread
(farrelta@ROG)~/Downloads
$ ./resource_pool.out
Resource Pool: 2 available
Requests: 5 threads
[Thread 1] Requesting resource...
[Thread 1] Resource acquired
[Thread 2] Requesting resource...
[Thread 2] Resource acquired
[Thread 3] Requesting resource...
[Thread 4] Requesting resource...
[Thread 5] Requesting resource...
[Thread 1] Released resource
[Thread 2] Released resource
```

3. In the resource pool program, comment out `sem_post(&pool)` and run it. Explain why the program eventually hangs: what happens to the semaphore's count, what state later threads enter when calling `sem_wait()`, and why no further progress is possible under counting-semaphore semantics.

Without “sem_post()”, the semaphore's count stays at 0. All later threads block in “sem_wait()”, and no further progress is possible under counting-semaphore semantics so the programs stuck.